FAKE NEWS DETECTOR – USER WORKFLOW

1. Input Stage (User Contribution)

- The **user submits news content** in one of these formats:
    - News **headline** or **article text**
    - **URL** of the article
    - **Social media post** (tweet, Facebook post, etc.)

Example: Paste a news link into the app or type a headline.

2. Processing Stage (Behind the Scenes)

- The system **cleans and organizes** the text:
    - Removes unnecessary parts (ads, punctuation, spam text).
    - Identifies the **main language** (English, Hindi, Telugu, etc.).
    - Breaks text into words for analysis.

Think of this like tidying up the news so the system can "read it" clearly.

3. Feature Extraction (Understanding the Content)

- The system converts text into a **mathematical form**:
    - Finds important words and phrases (TF-IDF).
    - Uses **AI language models (like BERT)** to capture meaning.
    - Analyzes writing style (length, grammar patterns, emotional tone).

This step is like turning news into "data" that the AI can analyze.

4. Model Analysis (Decision Making)

- Two layers of models check the news:
    - **Classical ML models** (Logistic Regression, SVM, Random Forest).
    - **Deep Learning models** (LSTM, Transformers like BERT).
- Results are **combined** (ensemble) for higher accuracy.

This is like getting multiple expert opinions before making a final call.

5. Post-Processing & Explainability

- The system **refines the decision**:
    - Adjusts based on confidence thresholds.

    - Checks if the **source is trustworthy**.
- Provides **explanations**:
    - Highlights suspicious words/sentences.
    - Shows which parts influenced the decision.

*This builds user trust — not just Fake/Real, but also **why**.*

6. Output to the User

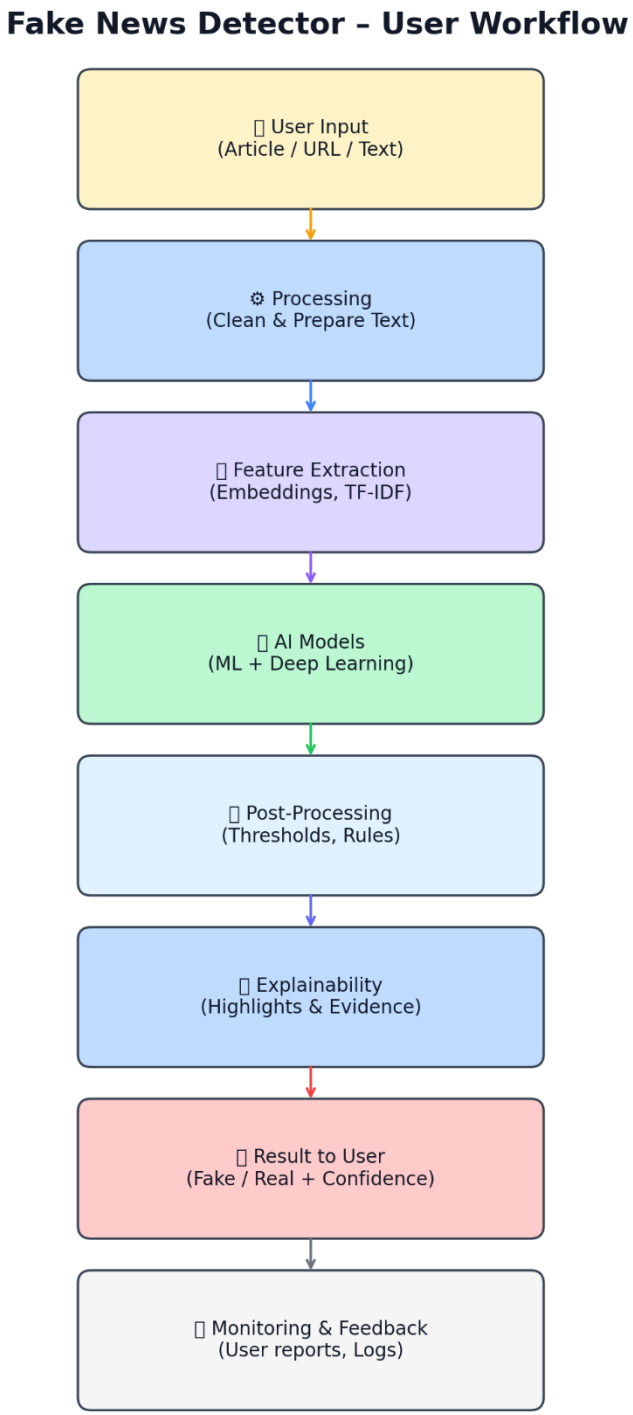
- The app shows:
    - Label: **Fake** or **Real**.
    - Confidence score (e.g., 92% sure this is Fake).
    - Highlighted evidence (keywords, sentences).

User clearly sees the result in a dashboard or mobile app.

7. Monitoring & Feedback

- Users can **give feedback** ("This is wrong/right").
- The system logs results for **future learning**.
- Helps improve the AI model continuously.

User participation makes the system smarter over time.



Backend Installations

requirements.txt

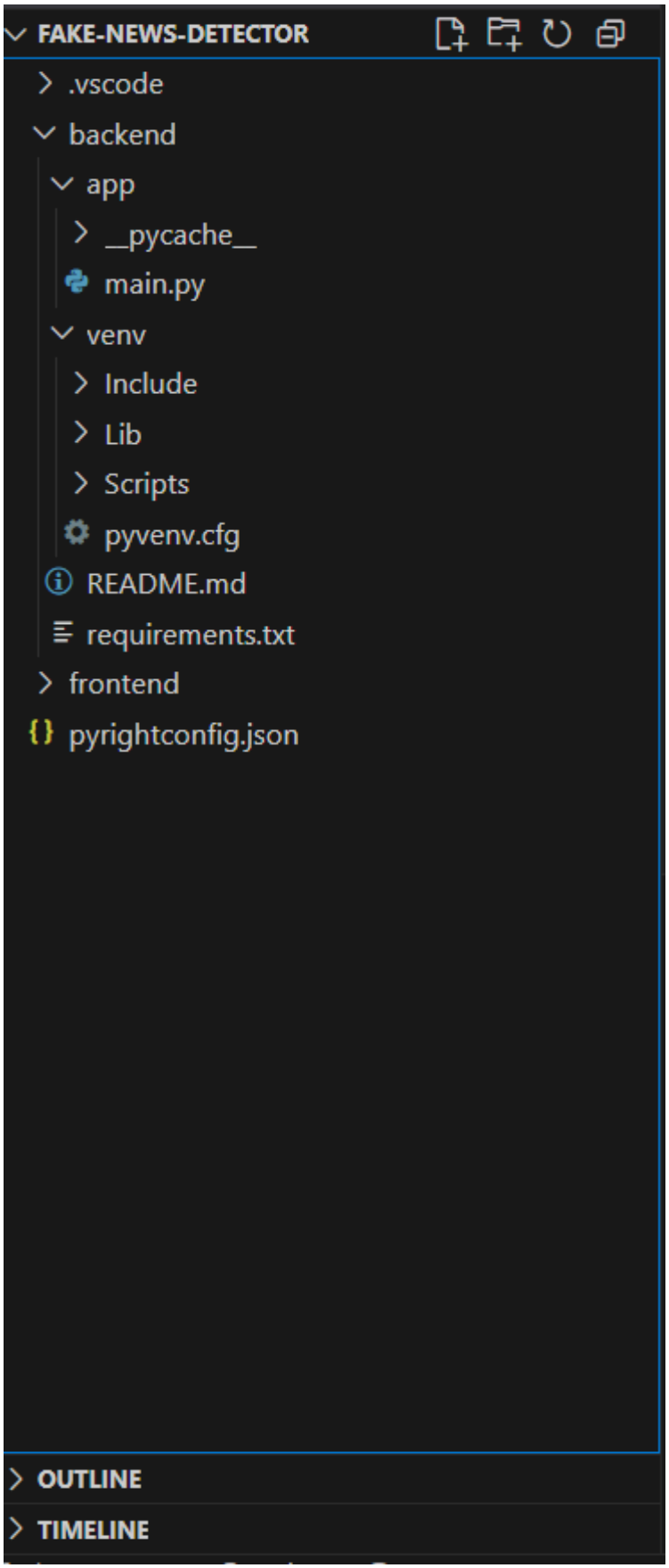
backend > requirements.txt

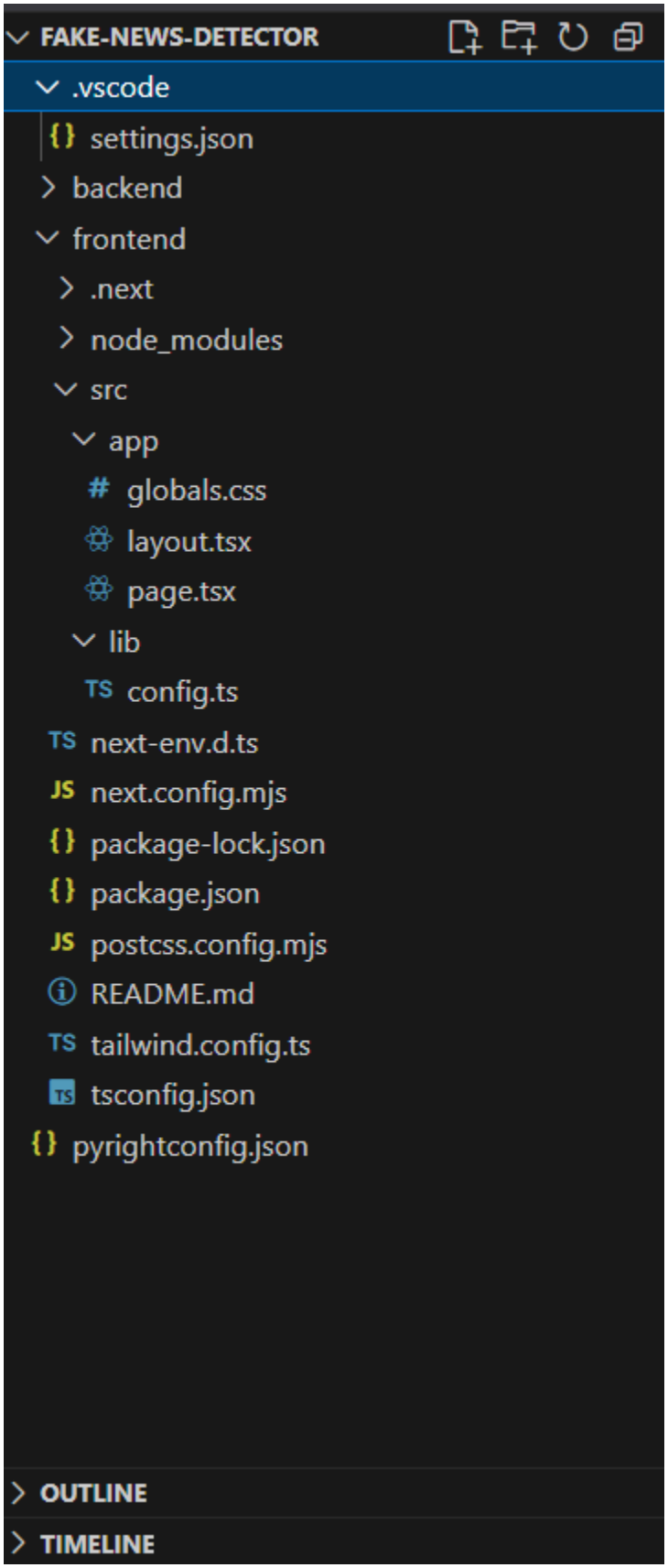
```
fastapi==0.115.2
uvicorn[standard]==0.30.6
requests==2.32.3
beautifulsoup4==4.12.3
```

Frontend Installations

```
npm install
added 16 packages
eslint@ 8.39,0: The flat rule is deprecated
 osdeFlatConfig is now available
added 243 packages
changed 9 packages
audited 259 packages in 6.507s
96 packages are looking for funding
run `npm fund' for details
13 moderate severrity vulnerabilities
run `npm audit fix
run npm auudit for details, run details
```

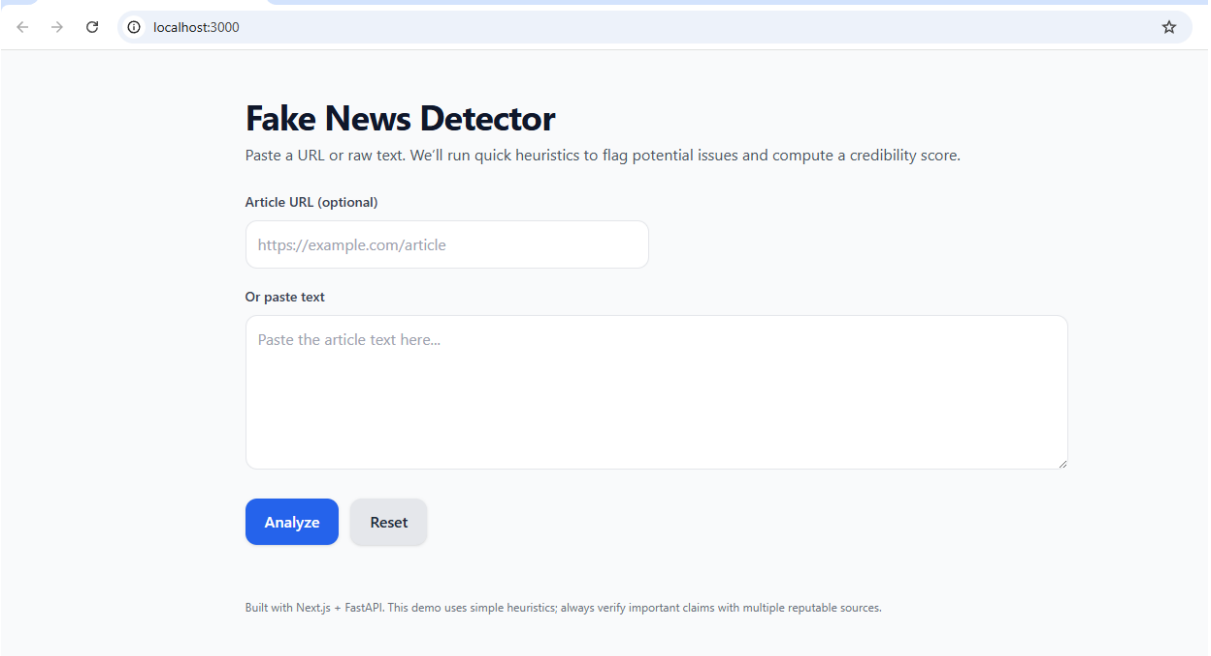
Project structure




FAKE-NEWS-DETECTOR
.vscode
settings.json
backend
frontend
.next
node_modules
src
app
globals.css
layout.tsx
page.tsx
lib
config.ts
next-env.d.ts
next.config.mjs
package-lock.json
package.json
postcss.config.mjs
README.md
tailwind.config.ts
tsconfig.json
pyrightconfig.json
OUTLINE
TIMELINE

Outputs:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   ···   uvicorn - backend

PS C:\Users\nikit\Downloads\All Projects\fake-news-detector> cd .\backend\
PS C:\Users\nikit\Downloads\All Projects\fake-news-detector\backend> uvicorn app.main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\nikit\\Downloads\\All Projects\\fake-news-detector\\backend']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [3820] using StatReload
INFO:     Started server process [2784]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\nikit\Downloads\All Projects\fake-news-detector> cd .\frontend\
PS C:\Users\nikit\Downloads\All Projects\fake-news-detector\frontend> npm run dev

> fake-news-detector-frontend@1.0.0 dev
> next dev

  ▲ Next.js 14.2.12
  - Local:        http://localhost:3000

 ✓ Starting...
 ✓ Ready in 6.3s
```